Wally Processor Cache Architecture Analysis

RISC-V Set-Associative L1 Cache Configuration

Cache Architecture Study

November 22, 2025

Contents

1 Default Parameters

The Wally processor implements a Harvard architecture with separate instruction and data caches. The following table presents the default cache configuration parameters for the `rv32gc` architecture:

Table 1: Wally Cache Configuration Parameters (rv32gc)

Parameter	**Value (default)**
DCACHE_NUMWAYS	4
DCACHE_WAYSIZEINBYTES	4096 bytes
DCACHE_LINELENINBITS	512 bits
CACHE_SRAMLEN	128
ICACHE_NUMWAYS	4
ICACHE_WAYSIZEINBYTES	4096 bytes
ICACHE_LINELENINBITS	512 bits

Both the instruction cache (I$) and data cache (D$) are implemented as **4-way set-associative L1 caches** with identical capacity and line size.

2 Question 1: Cache Size Calculation

Question

What is the size of the I$ cache and the D$ cache?

Calculation and Approach

The total cache size is computed as the product of the number of ways and the capacity per way:

$$\text{Total Cache Size} = \text{NUMWAYS} \times \text{WAYSIZEINBYTES}$$

For both I$ and D$:

$$\begin{aligned} \text{Size} &= 4 \text{ ways} \times 4096 \text{ bytes/way} \\ &= 16384 \text{ bytes} \\ &= 16 \text{ KiB} \end{aligned}$$

Answer

- **I$ (Instruction Cache) Size**: 16 KiB (16,384 bytes)
- **D$ (Data Cache) Size**: 16 KiB (16,384 bytes)

Note: Both caches have identical capacity in the default configuration, which is a common design pattern for symmetric systems.

3 Question 2: Instructions per Cache Line Fill

Question

Data is retrieved in blocks to the cache. How many instructions are updated into the I$ at once?

Calculation and Approach

On a single I$ cache miss, an entire cache line is fetched from memory. The number of instructions that fit in one cache line is determined by:

$$\text{Instructions per Line} = \frac{\text{ICACHE\_LINELENINBITS}}{\text{Instruction Width}}$$

From the configuration:

$$\begin{aligned}\text{Instructions per Line} &= \frac{512 \text{ bits}}{32 \text{ bits/instruction}} \\ &= 16 \text{ instructions}\end{aligned}$$

The 32-bit instruction width is the standard for the RV32 base RISC-V instruction set architecture (ISA).

Answer

- On a single I$ fill, **16 full 32-bit instructions** are brought into the cache simultaneously.
- If compressed 16-bit instructions (RVC extension) are used, a single cache line can accommodate up to 32 half-words, but the cache line itself remains 512 bits.

4 Question 3: Data Words per D$ Cache Fill

Question

Given that one data word is 32 bits, how many 32-bit words are brought into the D$ on a single cache fill?

Calculation and Approach

On a single D$ cache miss, the processor fetches an entire cache line from the next level of memory. The number of 32-bit data words per cache line is:

$$\text{Words per Cache Line} = \frac{\text{DCACHE\_LINELENINBITS}}{\text{Word Width}}$$

Substituting values:

$$\begin{aligned}\text{Words per Cache Line} &= \frac{512 \text{ bits}}{32 \text{ bits/word}} \\ &= 16 \text{ words}\end{aligned}$$

Answer

- **16 32-bit words** are brought into the D$ cache on a single cache line fill.
- In hexadecimal, this represents 64 bytes of data per cache fill (16 words × 4 bytes/word = 64 bytes).

5 Question 4: Cache Hit and Cache Miss

Question

When does a cache miss and a cache hit occur?

Definitions

Cache Hit

A **cache hit** occurs when:

1. The memory address requested by the processor maps to a cache line in the corresponding set.
2. The *tag bits* of the requested address match the stored tag in that cache line.
3. The *valid bit* of the cache line is set (indicating the line contains valid data).

When a cache hit is detected, the requested data can be returned directly from the cache with *low latency* (typically 1–2 cycles in L1 caches), avoiding a slow access to main memory.

Cache Miss

A **cache miss** occurs when:

1. The tag bits do not match any valid entry in the corresponding cache set (tag mismatch), *or*
2. The valid bit is not set for the matching tag (the cache line does not contain valid data).

On a cache miss, the processor must fetch the requested cache line from the next level in the memory hierarchy (e.g., L2 cache, main memory, or external memory). If all ways in the set are occupied, a *victim line* is selected (typically using an LRU replacement policy) and evicted. The new line is then brought into the cache.

References

- **Primary Reference**: Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design: The Hardware/Software Interface* (RISC-V Edition). Morgan Kaufmann. Chapter on Memory Hierarchy and Caches — provides formal definitions of cache hits, misses, miss penalty, and hit time.
- **Secondary Reference**: Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann. Sections 2.1–2.3 (Memory Hierarchy and Caches) cover cache behavior and performance metrics.

- **Online Reference**: Wikipedia, *CPU Cache*. `https://en.wikipedia.org/wiki/CPU_cache` — provides a concise conceptual overview of cache operations.

6 Question 5: Write-Back Strategy

Question

Wally RTL design follows the write-back strategy to provide cache-memory consistency. Briefly explain what this strategy is and how it differs from write-through.

Write-Back Strategy

Overview

The Wally processor uses a **write-back** (also called *copy-back*) cache write policy for the D$ to optimize performance. In this strategy:

1. When the processor executes a *store* (write) instruction, the data is written only to the cache line.
2. The cache line is marked as *dirty*, indicating that it contains data not yet synchronized with main memory.
3. The actual write to main memory is *deferred* until the dirty cache line is evicted (replaced) from the cache.

Advantages of Write-Back

- **Reduced memory bandwidth**: Multiple writes to the same cache line result in only one write to memory, exploiting temporal locality.
- **Lower write latency**: Writes are typically serviced at cache speed ($\sim$1–2 cycles) rather than memory speed (tens of cycles).
- **Better performance**: Systems with high write bandwidth or bursty write patterns benefit significantly.

Disadvantages of Write-Back

- **Complexity**: Requires dirty-bit tracking for each cache line and coordination of write-backs during evictions.
- **Coherence challenges**: In multi-core or multi-level cache systems, maintaining cache coherence becomes more complex because memory may not immediately reflect the latest values.
- **Data loss risk**: If the system loses power or crashes, uncommitted writes in the cache may be lost.

Write-Through Strategy (Comparison)

Overview

In a **write-through** policy:

1. When a store instruction executes, the data is written *simultaneously* to both the cache line and main memory (or the next level).
2. No dirty bit is needed; every write is immediately propagated to lower levels.

Advantages of Write-Through

- **Simpler coherence**: Memory always holds the latest value, simplifying coherence in multi-core systems.
- **Simpler correctness reasoning**: No dirty lines to track or synchronize.

Disadvantages of Write-Through

- **High memory bandwidth**: Every write generates a memory write, potentially saturating the memory bus.
- **High write latency**: Writes must wait for memory response, slowing down the processor (unless buffered).
- **Poor performance**: Systems with frequent writes or poor write locality suffer reduced performance.

Summary Table

Table 2: Write-Back vs. Write-Through Comparison

Characteristic	**Write-Back**	**Write-Through**
Memory writes on store	Deferred (on eviction)	Immediate
Dirty bit needed?	Yes	No
Memory bandwidth	Low	High
Write latency	Low	High
Coherence complexity	High	Low
Performance	Better (typical)	Worse (typical)

References

- **Primary Reference**: Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design: The Hardware/Software Interface* (RISC-V Edition). Morgan Kaufmann. Chapter 5 discusses cache write policies in detail.
- **Secondary Reference**: Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann. Sections on write policies and write-back/write-through tradeoffs.

- **Online References**:
  - Wikipedia, *Write-Back Cache*: `https://en.wikipedia.org/wiki/Write-back_cache`
  - Wikipedia, *Write-Through Cache*: `https://en.wikipedia.org/wiki/Write-through_cache`

7 Bonus: L2 Cache Design Choices

Question

What design choices are to be made if we would like to introduce an additional L2 cache into the Wally RTL design?

Key Design Decisions

1. Capacity, Associativity, and Line Size

- **Capacity**: Choose an L2 size larger than L1 (e.g., 64 KiB, 128 KiB, or 256 KiB). Must be a power of two for indexing.
- **Associativity**: Typically 8-way to 16-way for L2 (higher than L1) to reduce conflict misses.
- **Line Size**: L2 line size should match or be a multiple of L1 line size (e.g., 64 bytes or 128 bytes). If larger, L1 must handle sub-line fills.
- **Configuration**: Extend `config/rv32gc/config.vh` with new parameters like `L2CACHE_NUMWAYS`, `L2CACHE_CAPACITY`, `L2CACHE_LINELEN`, etc.

2. L1–L2 Interface and Protocol

- **Request/Response channels**: Define signals for L1 miss requests, L2 acknowledgments, data responses, and writebacks.
- **MSHRs (Miss Status Holding Registers)**: Add storage in L1 to track outstanding L2 requests, allowing L1 to accept new misses while waiting for fills.
- **Write-back channel**: L1 can send evicted dirty lines to L2 asynchronously or synchronously.
- **Priority/arbitration**: Decide how I$ and D$ share the L2 interface (e.g., fixed priority, round-robin, or QoS-based).

3. Replacement and Coherence Policy

- **Replacement**: Use LRU or pseudo-LRU similar to L1. For larger L2, consider approximate LRU or other policies.
- **Write policy**: Decide if L2 is write-back (preferred) or write-through to main memory.
- **Inclusion**: Choose between:

  - *Inclusive*: Every L1 line is guaranteed to exist in L2 (simpler coherence, more memory used).
  - *Exclusive*: L1 and L2 do not overlap (more complex, better memory utilization).
  - *Non-inclusive non-exclusive*: Most flexible but requires careful tracking.

4. Memory System Integration

- **Bus/interconnect**: L2 sits between L1 and main memory. Decide on the interconnect (AHB, AXI, or custom).
- **Latency paths**: Ensure L1 cache FSM can handle variable L2 latency (multiple cycles for L2 fill).
- **Arbitration**: In multi-core systems, multiple L1 instances (I$ and D$ per hart) share L2; implement arbitration logic.

5. Implementation Considerations

- **SRAM sizing**: L2 SRAM is much larger; may require multi-ported or banked structures for parallelism.
- **Tag and data separation**: L2 tags are larger (lower address bits indexed into L1, higher bits indexed into L2); careful address decoding needed.
- **Write-back buffers**: Implement temporal buffering between L1 evictions and L2 intake to smooth bursty writebacks.
- **Prefetching**: L2 can prefetch cache lines into L1 or directly service requests.

Code Locations to Modify in Wally Repository

Configuration

- `config/rv32gc/config.vh`: Add L2 cache parameters.

Cache and Memory Control

- `src/cache/cache.sv`: Modify or extend to interface with L2 instead of directly to main memory.
- `src/cache/cachefsm.sv`: Update FSM to handle L2 request/response handshakes and longer fill latencies.
- `src/cache/cacheLRU.sv`: May need re-parametrization for L2 associativity.
- `src/cache/cacheway.sv`: Adapt for L2 (larger tag width, more ways).
- Create new module `src/cache/l2cache.sv`: Implement L2 controller with FSM, SRAM management, tag comparison, and dirty-bit tracking.

LSU Integration

- `src/lsu/lsu.sv`: Update memory request path to route through L2; verify data widths and latency assumptions.
- Related LSU helpers (`align.sv`, `atomic.sv`, etc.): Ensure compatibility with L2 interface.

Top-Level Integration

- Top-level module (e.g., `cvw.sv` or `src/wally/top.sv`): Instantiate L2 cache controller and connect it to L1 instances, main memory interface, and AHB/interconnect.

Relevant Code Examples from Wally Repository

The following files in the OpenHW Wally repository serve as templates or references:

- **Configuration parameters**:
  - `https://raw.githubusercontent.com/openhwgroup/cvw/master/config/rv32gc/config.vh`
- **Cache implementation**:
  - `https://github.com/openhwgroup/cvw/blob/main/src/cache/cache.sv` — Top-level cache with FSM coordination.
  - `https://github.com/openhwgroup/cvw/blob/main/src/cache/cachefsm.sv` — FSM managing miss/fill/eviction.
  - `https://github.com/openhwgroup/cvw/blob/main/src/cache/cacheLRU.sv` — LRU replacement logic (good reference for L2).
  - `https://github.com/openhwgroup/cvw/blob/main/src/cache/cacheway.sv` — Per-way storage and tag comparison.
  - `https://github.com/openhwgroup/cvw/blob/main/src/cache/subcachelineread.sv` — Sub-line word multiplexing.
- **LSU integration**:
  - `https://github.com/openhwgroup/cvw/blob/main/src/lsu/lsu.sv` — Primary user of D$ and gateway to memory system.
  - `https://github.com/openhwgroup/cvw/tree/main/src/lsu` — Directory with LSU helper modules (align, atomic, etc.).

Implementation Notes

1. **Design effort**: Adding L2 requires RTL implementation of L2 controller, FSM extensions, configuration parameters, and verification.
2. **Testing**: Create test benches in `tests/` or `testbench/` to exercise L1–L2 interactions, multi-level fills, evictions, and write-back behavior.

3. **Performance simulation**: Use cycle-accurate simulation to evaluate hit rates, latencies, and bandwidth utilization of the multi-level hierarchy.

4. **Configuration updates**: `config.vh` must reflect the new L2 parameters so that Wally builds and runs correctly with L2 support.

References

[1] Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design: The Hardware/Software Interface* (RISC-V Edition). Morgan Kaufmann.

[2] Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.

[3] Wikipedia Contributors. (2024). "CPU Cache". *Wikipedia, The Free Encyclopedia*. Retrieved from `https://en.wikipedia.org/wiki/CPU_cache`

[4] Wikipedia Contributors. (2024). "Write-Back Cache". *Wikipedia, The Free Encyclopedia*. Retrieved from `https://en.wikipedia.org/wiki/Write-back_cache`

[5] Wikipedia Contributors. (2024). "Write-Through Cache". *Wikipedia, The Free Encyclopedia*. Retrieved from `https://en.wikipedia.org/wiki/Write-through_cache`

[6] Open Hardware Group. (2024). "CORE-V Wally: Configurable RISC-V Processor". GitHub Repository. Retrieved from `https://github.com/openhwgroup/cvw`